

CO4-AT1-Industrial Problem Solving

NAME: V Rakshana

SLOT: B

REGNO :192425137

COURSE CODE: CSA0911

QUESTION-1:

CODE:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class CustomerFeedback extends JFrame implements ActionListener {
    JLabel nameLabel, emailLabel, ratingLabel, feedbackLabel, resultLabel;
    JTextField nameField, emailField;
    JTextArea feedbackArea;
    JRadioButton r1, r2, r3, r4, r5;
    JButton submitButton, clearButton;

    CustomerFeedback() {
        setTitle("Customer Feedback System");
        setSize(600, 550);
        setLayout(new BorderLayout(10, 10));
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        JLabel title = new JLabel(
            "CUSTOMER FEEDBACK FORM",
            JLabel.CENTER);

        title.setFont(new Font("Arial", Font.BOLD, 22));
        add(title, BorderLayout.NORTH);

        JPanel formPanel = new JPanel();
        formPanel.setLayout(new GridLayout(5, 2, 10, 10));

        nameLabel = new JLabel("Customer Name:");
        nameField = new JTextField();
```

```
emailLabel = new JLabel("Email:");
emailField = new JTextField();
ratingLabel = new JLabel("Rating:");
JPanel ratingPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
r1 = new JRadioButton("1");
r2 = new JRadioButton("2");
r3 = new JRadioButton("3");
r4 = new JRadioButton("4");
r5 = new JRadioButton("5");
ButtonGroup group = new ButtonGroup();
group.add(r1);
group.add(r2);
group.add(r3);
group.add(r4);
group.add(r5);
ratingPanel.add(r1);
ratingPanel.add(r2);
ratingPanel.add(r3);
ratingPanel.add(r4);
ratingPanel.add(r5);
feedbackLabel = new JLabel("Feedback:");
feedbackArea = new JTextArea(5, 30);
feedbackArea.setLineWrap(true);
feedbackArea.setWrapStyleWord(true);
JScrollPane scrollPane = new JScrollPane(feedbackArea);
formPanel.add(nameLabel);
formPanel.add(nameField);
formPanel.add(emailLabel);
formPanel.add(emailField);
formPanel.add(ratingLabel);
```

```

formPanel.add(ratingPanel);
formPanel.add(feedbackLabel);
formPanel.add(scrollPane);
formPanel.add(new JLabel(""));
JPanel buttonPanel = new JPanel();
submitButton = new JButton("Submit Feedback");
clearButton = new JButton("Clear");
submitButton.addActionListener(this);
clearButton.addActionListener(this);
buttonPanel.add(submitButton);
buttonPanel.add(clearButton);
formPanel.add(buttonPanel);
add(formPanel, BorderLayout.CENTER);
resultLabel = new JLabel(
    "Enter your feedback and click Submit.",
    JLabel.CENTER);
resultLabel.setFont(new Font("Arial", Font.BOLD, 14));
add(resultLabel, BorderLayout.SOUTH);
setVisible(true);}

public void actionPerformed(ActionEvent e) {
    if (e.getSource() == submitButton) {
        String name = nameField.getText().trim();
        String email = emailField.getText().trim();
        String feedback = feedbackArea.getText().trim();
        if (name.isEmpty()) {
            JOptionPane.showMessageDialog(
                this,
                "Please enter customer name.",
                "Validation Error",
                JOptionPane.ERROR_MESSAGE);

```

```

        return;}

if (email.isEmpty() || !email.contains("@")) {
    JOptionPane.showMessageDialog(
        this,
        "Please enter a valid email address.",
        "Validation Error",
        JOptionPane.ERROR_MESSAGE);
    return;}

int rating = 0;

if (r1.isSelected())
    rating = 1;
else if (r2.isSelected())
    rating = 2;
else if (r3.isSelected())
    rating = 3;
else if (r4.isSelected())
    rating = 4;
else if (r5.isSelected())
    rating = 5;

if (rating == 0) {
    JOptionPane.showMessageDialog(
        this,
        "Please select a rating.",
        "Validation Error",
        JOptionPane.ERROR_MESSAGE);
    return; }

if (feedback.isEmpty()) {
    JOptionPane.showMessageDialog(
        this,
        "Please enter your feedback.",

```

```

        "Validation Error",
        JOptionPane.ERROR_MESSAGE);
    return;}
resultLabel.setText(
    "Feedback submitted successfully! Rating: "
    + rating + "/5");
JOptionPane.showMessageDialog(
    this,
    "Thank you, " + name + "!\n\n"
    + "Email: " + email + "\n"
    + "Rating: " + rating + "/5\n"
    + "Feedback: " + feedback,
    "Feedback Result",
    JOptionPane.INFORMATION_MESSAGE
);
}
else if (e.getSource() == clearButton) {
    nameField.setText("");
    emailField.setText("");
    feedbackArea.setText("");

    r1.setSelected(false);
    r2.setSelected(false);
    r3.setSelected(false);
    r4.setSelected(false);
    r5.setSelected(false);
    resultLabel.setText(
        "Enter your feedback and click Submit."
    );
}

```

```
}
```

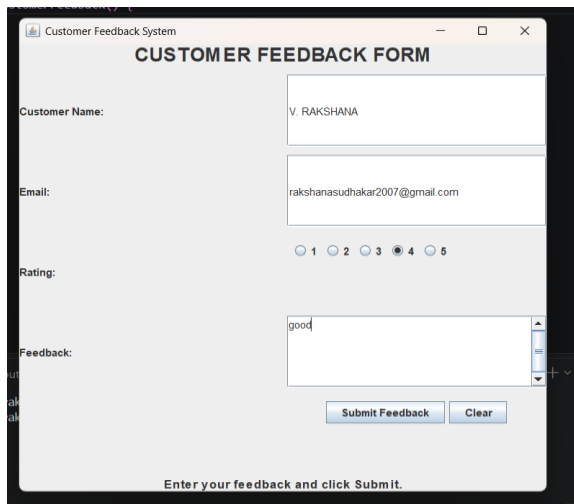
```
public static void main(String[] args) {
```

```
    new CustomerFeedback();
```

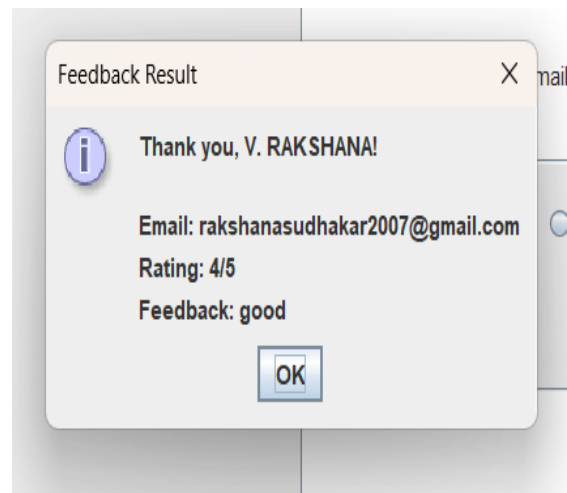
```
}
```

```
}
```

OUTPUT:



The screenshot shows a Java Swing window titled "Customer Feedback System". Inside the window is a form titled "CUSTOMER FEEDBACK FORM". The form has four input fields: "Customer Name:" with the text "V. RAKSHANA", "Email:" with the text "rakshanasudhakar2007@gmail.com", "Rating:" with radio buttons for 1, 2, 3, 4, and 5 (radio 4 is selected), and "Feedback:" with the text "good". Below the input fields are two buttons: "Submit Feedback" and "Clear". At the bottom of the window, there is a label that says "Enter your feedback and click Submit."



CO4-AT1-Industrial Problem Solving

NAME: V Rakshana

SLOT: B

REGNO :192425137

COURSE CODE: CSA0911

QUESTION-1:

CODE:

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;

public class HospitalAppointmentSystem extends JFrame implements ActionListener {
    JTextField patientField;
    JTextField doctorField;
    JTextField dateField;
    JTextField timeField;
    JButton bookButton;
    JButton updateButton;
    JButton cancelButton;
    JButton clearButton;
    JTable appointmentTable;
    DefaultTableModel tableModel;
    JMenuItem exitItem;
    JMenuItem clearAllItem;
    JMenuItem aboutItem;

    HospitalAppointmentSystem() {
        setTitle("Hospital Patient Appointment Management System");
        setSize(850, 600);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        JMenuBar menuBar = new JMenuBar();
```

```
JMenu fileMenu = new JMenu("File");
JMenu helpMenu = new JMenu("Help");
clearAllItem = new JMenuItem("Clear All Appointments");
exitItem = new JMenuItem("Exit");
aboutItem = new JMenuItem("About");
fileMenu.add(clearAllItem);
fileMenu.addSeparator();
fileMenu.add(exitItem);
helpMenu.add(aboutItem);
menuBar.add(fileMenu);
menuBar.add(helpMenu);
setJMenuBar(menuBar);
clearAllItem.addActionListener(this);
exitItem.addActionListener(this);
aboutItem.addActionListener(this);
JPanel mainPanel = new JPanel(new BorderLayout(10, 10));
JLabel titleLabel = new JLabel(
    "HOSPITAL APPOINTMENT MANAGEMENT SYSTEM",
    JLabel.CENTER);
titleLabel.setFont(new Font("Arial", Font.BOLD, 22));
mainPanel.add(titleLabel, BorderLayout.NORTH);
JPanel inputPanel = new JPanel(new GridLayout(4, 2, 10, 10));
inputPanel.setBorder(
    BorderFactory.createTitledBorder("Patient Appointment Details"));
inputPanel.add(new JLabel("Patient Name:"));
patientField = new JTextField();
inputPanel.add(patientField);
inputPanel.add(new JLabel("Doctor Name:"));
doctorField = new JTextField();
```



```

inputPanel.add(doctorField);
inputPanel.add(new JLabel("Appointment Date:"));
dateField = new JTextField();
inputPanel.add(dateField);
inputPanel.add(new JLabel("Appointment Time:"));
timeField = new JTextField();
inputPanel.add(timeField);
JPanel buttonPanel = new JPanel(new FlowLayout());
bookButton = new JButton("Book Appointment");
updateButton = new JButton("Update Appointment");
cancelButton = new JButton("Cancel Appointment");
clearButton = new JButton("Clear Fields");
buttonPanel.add(bookButton);
buttonPanel.add(updateButton);
buttonPanel.add(cancelButton);
buttonPanel.add(clearButton);
bookButton.addActionListener(this);
updateButton.addActionListener(this);
cancelButton.addActionListener(this);
clearButton.addActionListener(this);
JPanel topPanel = new JPanel(new BorderLayout(10, 10));
topPanel.add(titleLabel, BorderLayout.NORTH);
topPanel.add(inputPanel, BorderLayout.CENTER);
topPanel.add(buttonPanel, BorderLayout.SOUTH);
mainPanel.add(topPanel, BorderLayout.NORTH);
String[] columns = {
    "Patient Name",
    "Doctor Name",
    "Date",
    "Time"};

```

```

tableModel = new DefaultTableModel(columns, 0);
appointmentTable = new JTable(tableModel);
appointmentTable.setSelectionMode(
    ListSelectionModel.SINGLE_SELECTION);
JScrollPane scrollPane = new JScrollPane(appointmentTable);
scrollPane.setBorder(
    BorderFactory.createTitledBorder("Appointment Schedule"));
mainPanel.add(scrollPane, BorderLayout.CENTER);
JLabel statusLabel = new JLabel(
    "Ready - Manage patient appointments",
    JLabel.CENTER);
mainPanel.add(statusLabel, BorderLayout.SOUTH);
add(mainPanel);
setVisible(true);
}

public void actionPerformed(ActionEvent e) {
    if (e.getSource() == bookButton) {
        String patient = patientField.getText().trim();
        String doctor = doctorField.getText().trim();
        String date = dateField.getText().trim();
        String time = timeField.getText().trim();
        if (patient.isEmpty() ||
            doctor.isEmpty() ||
            date.isEmpty() ||
            time.isEmpty()) {
            JOptionPane.showMessageDialog(
                this,
                "Please fill in all appointment details.",
                "Validation Error",

```

```

        JOptionPane.ERROR_MESSAGE
    );
    return;
}
tableModel.addRow(
    new Object[]{
        patient,
        doctor,
        date,
        time
    }
);
JOptionPane.showMessageDialog(
    this,
    "Appointment booked successfully!",
    "Confirmation",
    JOptionPane.INFORMATION_MESSAGE
);
clearFields();
}
else if (e.getSource() == updateButton) {
    int selectedRow =
        appointmentTable.getSelectedRow();
    if (selectedRow == -1) {
        JOptionPane.showMessageDialog(
            this,
            "Please select an appointment to update.",
            "Update Error",
            JOptionPane.WARNING_MESSAGE
        );
    }
}

```

```

        return;
    }
    String patient = patientField.getText().trim();
    String doctor = doctorField.getText().trim();
    String date = dateField.getText().trim();
    String time = timeField.getText().trim();
    if (patient.isEmpty() ||
        doctor.isEmpty() ||
        date.isEmpty() ||
        time.isEmpty()) {
        JOptionPane.showMessageDialog(
            this,
            "Please fill in all appointment details.",
            "Validation Error",
            JOptionPane.ERROR_MESSAGE
        );
        return;
    }
    tableModel.setValueAt(patient, selectedRow, 0);
    tableModel.setValueAt(doctor, selectedRow, 1);
    tableModel.setValueAt(date, selectedRow, 2);
    tableModel.setValueAt(time, selectedRow, 3);
    JOptionPane.showMessageDialog(
        this,
        "Appointment updated successfully!",
        "Update",
        JOptionPane.INFORMATION_MESSAGE
    );
    clearFields();
}

```

```

else if (e.getSource() == cancelButton) {
    int selectedRow =
        appointmentTable.getSelectedRow();
    if (selectedRow == -1) {
        JOptionPane.showMessageDialog(
            this,
            "Please select an appointment to cancel.",
            "Cancellation Error",
            JOptionPane.WARNING_MESSAGE
        );
        return;
    }
    int choice = JOptionPane.showConfirmDialog(
        this,
        "Are you sure you want to cancel this appointment?",
        "Confirm Cancellation",
        JOptionPane.YES_NO_OPTION
    );
    if (choice == JOptionPane.YES_OPTION) {
        tableModel.removeRow(selectedRow);
        JOptionPane.showMessageDialog(
            this,
            "Appointment cancelled successfully.",
            "Cancellation",
            JOptionPane.INFORMATION_MESSAGE
        );
        clearFields();
    }
}

```

```

else if (e.getSource() == clearButton) {
    clearFields();
}
else if (e.getSource() == clearAllItem) {
    if (tableModel.getRowCount() == 0) {
        JOptionPane.showMessageDialog(
            this,
            "There are no appointments to clear."
        );
        return;
    }
    int choice = JOptionPane.showConfirmDialog(
        this,
        "Clear all appointments?",
        "Confirm",
        JOptionPane.YES_NO_OPTION
    );
    if (choice == JOptionPane.YES_OPTION) {
        tableModel.setRowCount(0);
        JOptionPane.showMessageDialog(
            this,
            "All appointments have been cleared.");
    }
}
else if (e.getSource() == exitItem) {
    System.exit(0);
}
else if (e.getSource() == aboutItem) {

    JOptionPane.showMessageDialog(
        this,
        "Hospital Appointment Management System\n"

```

```

+ "Developed using Java Swing\n\n"
+ "Supports booking, updating and cancellation.",
"About",
JOptionPane.INFORMATION_MESSAGE));}}

void clearFields() {
    patientField.setText("");
    doctorField.setText("");
    dateField.setText("");
    timeField.setText("");
    appointmentTable.clearSelection();}

public static void main(String[] args) {
    SwingUtilities.invokeLater(
        new Runnable() {
            public void run() {
                new HospitalAppointmentSystem();}}}

```

OUTPUT:

